

Diseño de agentes basados en LLM

Taller intensivo de 3 horas

Rol · Herramientas · Memoria · Planificación (ReAct)

con LangGraph sobre Ollama Cloud

MUIA
Máster Universitario en Inteligencia Artificial

Escuela Superior de Informática
Universidad de Castilla-La Mancha
Ciudad Real

3 de octubre de 2026

Material del taller. Todas las libretas incluidas se han ejecutado y verificado sobre el entorno descrito en el anexo B.

Índice general

I Parte I. Guía del taller **5**

1. La idea que sostiene todo el seminario **7**

2. Diseño incremental: cada módulo añade una sola cosa **8**

3. Agenda (180 minutos) **9**

4. Cómo explicaría cada concepto **10**
4.1 Rol — «el rol es un parámetro, no una personalidad» . . . 10
4.2 Herramientas — «el modelo decide, el grafo ejecuta» . . . 10
4.3 Memoria — «un LLM no recuerda; le reenvías el historial» . . . 11
4.4 Planificación — «la planificación es un campo del estado» . . . 11

5. Qué no se enseña (y por qué) **13**

6. Puesta en marcha **14**
Cómo se regeneran las libretas . . . 14

7. Plan de contingencia **15**
Aviso legal para el aula . . . 15

8. Requisitos del alumnado **16**

9. Mapa de ficheros **17**

10. Evaluación **18**

II Parte II. Los seis módulos **19**

00 · Antes de empezar: comprobar el terreno **20**
Por qué dedicamos diez minutos a esto . . . 20
0.1 ¿Qué tengo instalado? . . . 20
0.2 ¿Qué modelos tengo? . . . 20
0.3 La prueba de fuego: ¿cuál responde? . . . 21
No todos los fallos son iguales . . . 21
0.4 Elegir el modelo del seminario . . . 21
0.5 ¿Responde bien a una petición normal? . . . 22
0.6 La prueba crítica: **¿sabe usar herramientas?** . . . 22
Si algo ha fallado . . . 22
Lo que hemos aprendido y seguiremos usando . . . 23

01 · El rol: quién es el agente **24**
El mapa del seminario . . . 24
1.1 El grafo más pequeño posible . . . 24
1.2 El rol es un parámetro, no una constante . . . 25
1.3 El rol da **contenido**, pero no garantiza la **forma** . . . 25
1.4 La forma se impone por el **canal de herramientas** . . . 26
1.5 Un rol que se puede **verificar** . . . 26
Resumen del módulo . . . 27
Ejercicios . . . 27

02 · Herramientas: el agente deja de inventar **28**
2.1 Definir herramientas y ver lo que ve el modelo . . . 28
2.2 El bucle ReAct, escrito a mano . . . 28
2.3 Ejecutarlo y, sobre todo, **leer la traza** . . . 29
2.4 Cuando una herramienta falla . . . 30
2.5 Todo esto, en una línea: `create_agent` . . . 31
Resumen del módulo . . . 31
Ejercicios . . . 32

03 · Memoria: continuidad y aprendizaje **33**
3.1 Sin checkpointer, el agente tiene amnesia . . . 33
3.2 Con checkpointer, hay hilo . . . 34
3.3 Hilos distintos, memorias distintas . . . 34
3.4 ¿Dónde está la memoria exactamente? . . . 34
3.5 Memoria a largo plazo: el `Store` . . . 34
3.6 La prueba que lo demuestra: memoria que cruza hilos . . . 35
3.7 Tres avisos que ahorran disgustos . . . 36
Resumen del módulo . . . 36
Ejercicios . . . 36

04 · Planificación: decidir antes de actuar **38**
4.1 ReAct bajo el microscopio . . . 38
4.2 Plan-and-execute: la planificación es **un campo más del estado** . . . 39
4.3 Intervención humana: parar el grafo a mitad . . . 41
Cuatro reglas de `interrupt()` que hay que interiorizar . . . 42
Resumen del módulo . . . 42
El mapa completo del seminario . . . 42
Ejercicios . . . 42

05 · Ampliaciones (material para casa) **44**
5.1 Persistencia: memoria en disco . . . 44
5.2 Streaming: no hacer esperar al usuario . . . 45
5.3 Supervisión: el rol, a escala . . . 45
5.4 Middleware: lo que no hemos escondido . . . 46
Reto integrador (para casa o para evaluar) . . . 47
Rúbrica . . . 47
La pregunta que importa . . . 47

III Anexos **48**

A. Evidencia de ejecución real **49**
A.1 Auditoría de modelos: retirados, fuera de cuota y utilizables 49
A.2 Tool calling confirmado sobre el modelo gratuito 49
A.3 La traza del bucle ReAct: el modelo pide, el grafo ejecuta 49
A.4 Memoria de largo plazo cruzando conversaciones distintas 49
A.5 Planificación explícita: el plan se lee antes de ejecutarse 50

B. Verificación del entorno **51**

C. Referencia rápida de la API **52**
Advertencias que ahorran tiempo 53

D. Estructura del material **54**

Parte I. Guía del taller

Rol · Herramientas · Memoria · Planificación (ReAct)
Taller intensivo de 3 horas · Ollama Cloud (plan gratuito) · Python

---

1. La idea que sostiene todo el seminario

La mayoría de los cursos de agentes enseñan a llamar a una API:

```
agente = initialize_agent(tools, llm, agent=AgentType.OPENAI_FUNCTIONS)
agente.invoke("...")
```

Eso funciona hasta que el agente se comporta raro. Entonces el alumno no sabe qué mirar, porque **la estructura del agente está escondida dentro de la librería**.

Este seminario hace lo contrario. Los cuatro conceptos de cualquier sistema agéntico dejan de ser magia y pasan a ser **objetos que se ven dibujados en el grafo**:

Concepto	Dónde vive en LangGraph	El alumno lo ve como
Rol	el `system_prompt` de un nodo	la función que él escribe
Herramientas	`ToolNode` + una arista condicional	un nodo y una flecha
Memoria	`checkpointer` + `thread_id` · `Store`	el `config` que pasa en `invoke`
Planificación	un campo `plan` y un contador en el estado	el bucle que dibuja con `draw_mermaid()`

La frase que resume el taller: en LangGraph la estructura del agente es un dato que se puede imprimir, revisar y modificar. Esa es la razón didáctica de usar LangGraph y no una cadena.

2. Diseño incremental: cada módulo añade una sola cosa

El seminario es una escalera. En cada peldaño se modifica el grafo del peldaño anterior en exactamente un aspecto, y siempre se ejecuta para ver el efecto:

```
Módulo 01  START -> agente -> END                    (rol)
Módulo 02  + ToolNode + arista condicional            (herramientas)  -> bucle ReAct
Módulo 03  + checkpointer + Store                     (memoria)       -> sin tocar el grafo
Módulo 04  + campo plan + contador + interrupt        (planificación) -> bucle controlado
```

Nada de «ahora abrimos el fichero final de 200 líneas». El alumno construye el agente completo y, al final, descubre que `create_agent` hace lo mismo en una línea. En ese orden: **primero la mecánica, después el atajo**, nunca al revés.

3. Agenda (180 minutos)

Hora	Módulo	Contenido	Min
0:00	**00 · Preflight**	Comprobar modelo, cuota y **tool calling**	15
0:15	**01 · Rol**	Estado, nodos, aristas y el rol como parámetro	30
0:45	**02 · Herramientas**	`@tool`, `ToolNode`, `tools_condition`, la traza	35
1:20	—	**Descanso**	10
1:30	**03 · Memoria**	`checkpointer` + `thread_id`, `Store` de largo plazo	30
2:00	**04 · Planificación**	ReAct por dentro, plan-and-execute, `interrupt`	35
2:35	**Cierre**	El mapa, `create_agent`, reto y rúbrica	25

El módulo 05 (persistencia, streaming, supervisor, middleware) es **material para casa**: no cabe en tres horas y no hace falta para entender lo básico.

4. Cómo explicaría cada concepto

Esta es la parte que no está en el código. Para cada concepto: la frase, la analogía, el experimento, el detalle que atasca y la pregunta de comprobación.

4.1 Rol — «el rol es un parámetro, no una personalidad»

La frase. Un rol no es «actúa como un chef». Un rol es *identidad + objetivo + límites + formato*, y solo lo último se puede verificar.

La analogía. El estado (`MessagesState`) es un **sobre** que va pasando de mesa en mesa. Un nodo no escribe una carta nueva: **añade una hoja al sobre** y lo devuelve. El rol es la primera hoja que metes: la carta de presentación.

El experimento (01). Se construye el grafo más pequeño posible (`START → llm → END`) y se le pregunta «¿Quién eres?». Responde como un asistente genérico. Después se fabrica el **mismo grafo** con dos roles —chef y nutricionista— y se les hace la misma pregunta. Las respuestas divergen. La conclusión es inmediata: *el rol no cambia el motor, cambia el sobre.*

El detalle que atasca. Que un nodo devuelve **el parche**, no el estado completo. Si devuelves el estado entero pisas lo que había. Se ve en la primera celda.

La segunda mitad del módulo, que es la importante. Se pide al modelo un JSON «por favor», tres veces, y se cuenta cuántas lo cumple. Falla. Entonces se cambia a `create_agent(..., response_format=MiModeloPydantic)` y funciona siempre. Mensaje: **la forma no se pide, se impone por el canal de herramientas**. Esto prepara el módulo 02 y explica de paso por qué muchos agentes «a veces» devuelven JSON y «a veces» no.

> Dato real de este entorno: Ollama Cloud **no soporta salida estructurada** oficialmente. Por eso el canal de herramientas no es un truco, es *la* solución correcta.

Pregunta de comprobación. «Si quieres que tu agente nunca proponga una receta de más de 15 minutos, ¿lo escribes en el prompt y ya está?» Respuesta esperada: no, se verifica en código, porque el prompt es una sugerencia.

4.2 Herramientas — «el modelo decide, el grafo ejecuta»

La frase. El modelo **no ejecuta nada**: pide que se ejecute. Eso es todo.

La analogía. El modelo es un jefe que rellena **impresos de petición** (`AIMessage.tool_calls`). El `ToolNode` es el bedel que los ejecuta y devuelve el resultado (`ToolMessage`). El jefe nunca toca la máquina.

El experimento (02). Se define una herramienta y se **imprime el JSON Schema** que el modelo recibe. Solo eso. El alumno descubre que el docstring *es* el prompt de la herramienta: un docstring vago hace una herramienta inservible. Después se monta el bucle a mano y se **lee la traza en voz alta**:

```
[00] HumanMessage      la pregunta
[01] AIMessage -> TOOL_CALLS: buscar_precio(...), convertir_moneda(...)
[02] ToolMessage       5.0
[03] ToolMessage       110.0
[04] AIMessage         la respuesta, ya con datos reales
```

El momento clave es el `[01]`: **el modelo no contesta, pide**. Y si esa línea no aparece en la traza, el modelo **se lo está inventando**. Ese es el síntoma que hay que enseñar a detectar, porque es exactamente el fallo del notebook original de este repositorio: el agente de *function calling* respondía con un tipo de cambio inventado y nadie se enteró.

El detalle que atasca. `tools_condition` devuelve los literales `"tools"` y `"__end__"`. Si tu nodo se llama `herramientas`, **no lo adivina**: hay que mapearlo en `add_conditional_edges(..., {"tools": "herramientas", "__end__": END})`. Sin eso, el error es `found unknown target 'tools'`. Merece la pena provocarlo en clase.

La traca final. `create_agent` construye el mismo grafo en una línea. Se imprimen los dos grafos juntos y se comparan: «esto que has escrito a mano es lo que hay dentro».

Pregunta de comprobación. «Tu agente dice que el pollo cuesta 12 € y tu base de datos dice 5 €. ¿Dónde miras primero?» Respuesta esperada: en la traza, buscando si hubo `tool_calls` o si el modelo se lo inventó.

4.3 Memoria — «un LLM no recuerda; le reenvías el historial»

La frase. La memoria no es una facultad del modelo: es un dato que alguien guarda y vuelve a enviar.

La analogía. El `checkpointer` es el **cuaderno de la conversación**: se identifica por `thread_id`, es decir, por *esta charla*. El `Store` es la **ficha del cliente**: se identifica por usuario y sobrevive a todas las charlas. Confundirlos es el error número uno.

El experimento (03), en cuatro actos.

1. Sin `checkpointer`, se dice «me llamo Luis» y luego se pregunta «¿cómo me llamo?». No lo sabe. Se cuentan los mensajes que ha visto el modelo en el turno 2: dos. El historial anterior se tiró a la basura.
2. Con `InMemorySaver` y `thread_id`, sí lo sabe. Ahora hay cuatro mensajes.
3. Con **otro** `thread_id`, vuelve a no saberlo. El aislamiento es gratis.
4. Se abre el checkpoint con `get_state(config)` y **se imprimen los mensajes guardados**. Ahí se desmitifica del todo: la memoria es una lista.

Después, el `Store` con una herramienta que lee y otra que escribe, y la prueba decisiva: **guardar en el hilo 1 y preguntar en el hilo 2**. El `checkpointer` no puede ayudar porque son conversaciones distintas. Si lo recuerda, es del `Store`.

El detalle que atasca. Olvidar el `checkpointer` **no da error**: simplemente no hay memoria. Es el fallo silencioso más caro. Se provoca a propósito.

Pregunta de comprobación. «Tu agente recuerda tu nombre en una conversación pero no en otra. ¿Es un bug?» Respuesta esperada: no, es `thread_id` distinto; para eso está el `Store`.

4.4 Planificación — «la planificación es un campo del estado»

La frase. ReAct decide **un paso cada vez**, mirando solo lo que ya ha pasado. Es reactivo, no planificado.

La analogía. ReAct es salir de casa sin lista y volver al supermercado tres veces. Plan-and-execute es escribir la lista antes de salir. Cuesta lo mismo escribirla, pero te ahorra viajes… y te permite **enseñársela a alguien antes de gastar dinero**.

El experimento (04).

1. **ReAct bajo el microscopio.** Una tarea que obliga a encadenar (precio → conversión). Se cuentan las vueltas del bucle: cada vuelta es una llamada al modelo, y el agente no sabía al empezar cuántas haría.

2. **Plan-and-execute.** Se añade un nodo `planificar` que devuelve una lista con salida estructurada, un nodo `ejecutar` que consume `plan[paso]`, un campo `paso` que actúa de contador, y una arista condicional que vuelve a `ejecutar` mientras queden pasos. Se imprime el plan **y** la traza de resultados.

El momento didáctico: el alumno ve que planificar no requiere ningún framework nuevo, solo **un campo más en el estado y un contador**. Eso es todo. Y la estructura es legible antes de ejecutarse, que es la ventaja real.

3. **El freno humano.** `interrupt()` pausa el grafo y `Command(resume=...)` lo reanuda. Se ejecuta la pausa, se lee la propuesta en pantalla, el humano responde y el grafo continúa donde estaba.

El detalle que atasca. `interrupt()` **reejecuta el nodo entero** al reanudar. Todo lo que haya antes se vuelve a ejecutar: los efectos secundarios deben ser idempotentes. Y nunca dentro de un `while` ni de un `try/except` desnudo.

Pregunta de comprobación. «¿Quién es responsable si el plan está mal, el modelo o tú?»

5. Qué no se enseña (y por qué)

Enseñar una API obsoleta es el error que cometía el material de partida de este repositorio, cuyo propio output incluía:

```
LangChainDeprecationWarning: ... new use cases to be built with LangGraph
```

Por eso, deliberadamente:

No se enseña	Se enseña en su lugar	Motivo
`initialize_agent` + `AgentType`	`create_agent`	Obsoleto desde LangChain 1.0
`create_react_agent` (`langgraph.prebuilt`)	`create_agent` (`langchain.agents`)	Emite `LangGraphDeprecationWarning`; retirada prevista en v2
`MemorySaver`	`InMemorySaver`	Alias equivalente, pero no canónico
`from langchain.prompts import ...`	`langchain_core` / `langchain.messages`	Rutas movidas
`ConversationChain` y familia	`checkpointer` + `thread_id`	Arquitectura anterior
`tool_choice`	prompting	**Ollama no lo soporta**
Salida estructurada vía `json_schema`	`response_format=` con Pydantic	**La nube no la soporta**; `method="function_calling"` sí funciona

La versión de referencia está en `referencia/api-2026.md`, verificada contra los *wheels* publicados, no contra blogs.

6. Puesta en marcha

```
cd seminario
uv venv --python 3.12 .venv          # 3.11-3.13: LangGraph no declara soporte para 3.14
uv pip install --python .venv/bin/python -r requirements.txt
.venv/bin/python -m ipykernel install --user --name seminario-langgraph \
    --display-name "Seminario LangGraph"
```

Y **diez minutos antes de clase**, el verificador:

```
.venv/bin/python verificar_entorno.py
```

Comprueba daemon, modelo, tool calling, salida estructurada, memoria, store e interrupción, y dice exactamente qué hacer si algo falla.

Cómo se regeneran las libretas

Las libretas **no se editan a mano**. La fuente son ficheros Python en `_build/`, y el generador las construye **y las ejecuta**:

```
.venv/bin/python _build/build.py              # todas
.venv/bin/python _build/build.py nb_02 nb_03  # solo algunas
```

Si una celda falla, el build falla. Así el alumno recibe salidas reales, no promesas.

7. Plan de contingencia

El plan gratuito de Ollama tiene límites reales. Está todo previsto:

Síntoma	Causa	Qué hacer
`HTTP 402 not included in your free usage`	Ese modelo no está en el plan gratuito	`comun.py` ya prueba varios; usa `gemma4:31b-cloud`
`HTTP 410 was retired at ...`	El proveedor retiró el modelo	Ya ocurrió con `qwen3.5:cloud` (2026-09-25). Cambia de modelo
`HTTP 429`	Cuota mensual agotada	**Retroceso a local**: `SEMINARIO_MODELO=gemma4:e4b`
Peticiones que se encolan	El plan gratuito permite **1 petición concurrente**	Diseña las demos en secuencia; nada de *fan-out*
`HTTP 400` con esquemas de herramientas	La nube **rechaza `oneOf`**	Simplifica el esquema de la herramienta
El modelo no llama a las herramientas	Modelo sin capacidad `tools`	`ollama show <modelo>` y comprueba `tools`; usa `gemma4:e4b`
Todo falla	Sin internet o sin cuota	`gemma4:e4b` ya está en local y **hace tool calling**

Modelos verificados como gratuitos y funcionales en este entorno (2026-10-03): `gemma4:31b-cloud`, `gemma4:cloud`, `gpt-oss:20b-cloud`, `gpt-oss:120b-cloud`, `nemotron-3-super:cloud`, `nemotron-3-nano:30b-cloud`, `nemotron-3-ultra:cloud`, y el local `gemma4:e4b`.

Modelos que fallan (y son una lección, no un problema): 11 con `402` de cuota (`glm-5.2`, `kimi-k3`, `minimax-m3`, `deepseek-v4-pro`…) y 2 retirados (`qwen3.5`). Esto se ve en vivo en la libreta 00 y es el mejor argumento para el preflight.

Aviso legal para el aula

Los términos de Ollama exigen **18 años** y **una cuenta por persona**. Cada alumno debe crear su propio *signin* o su propia API key: no compartas una clave de clase ni automatices la creación de cuentas.

8. Requisitos del alumnado

- Python a nivel de funciones, diccionarios y `try/except`. No hace falta nada de ML.
- **Ningún conocimiento previo de LangChain ni de LangGraph.** El módulo 01 empieza por «qué es un nodo».
- Un portátil con Ollama instalado. Alternativa sin instalación: una API key de `https://ollama.com/settings/keys` y `openai` apuntando a `https://ollama.com/v1`.

9. Mapa de ficheros

```
seminario/├──
 README.md                  <- esta guía docente├──
 00_preflight.ipynb         <- comprobar el terreno (15 min)├──
 01_rol.ipynb               <- estado, nodos, aristas y el rol├──
 02_herramientas.ipynb      <- tools y el bucle ReAct a mano├──
 03_memoria.ipynb           <- checkpointer y Store├──
 04_planificacion.ipynb     <- ReAct por dentro, plan-and-execute, interrupt├──
 05_ampliaciones.ipynb      <- para casa: SQLite, streaming, supervisor├──
 comun.py                   <- fuente única del modelo y utilidades de clase├──
 verificar_entorno.py       <- comprobación completa antes de clase├──
 requirements.txt├──
 .env.example├──
 _build/                    <- fuentes de las libretas + generador└──
 referencia/                <- API verificada y capacidades de Ollama Cloud
```

10. Evaluación

La rúbrica completa está al final de la libreta 05. El criterio de fondo es uno:

> **¿En qué punto exacto de tu grafo tomaría una decisión equivocada el agente, y cómo te enterarías?**

Si el alumno sabe responder eso, ha entendido el seminario. Si no, ha aprendido a invocar una API.

Parte II. Los seis módulos

00 · Antes de empezar: comprobar el terreno

Objetivo: salir de esta libreta sabiendo *qué modelo vas a usar y por qué*, y con la certeza de que tu entorno soporta **tool calling**, que es la pieza sin la cual nada de lo que viene después funciona.

Por qué dedicamos diez minutos a esto

Un modelo puede aparecer en `ollama list` y estar perfectamente inservible. No es un caso teórico: al preparar este seminario, en esta misma máquina, nos encontramos con

modelo listado	lo que pasa de verdad
`qwen3.5:cloud`	`was retired at 2026-09-25` — el proveedor lo retiró
`glm-5.2:cloud`	`not included in your free usage` — fuera del plan gratuito
`gemma4:31b-cloud`	funciona, y además hace tool calling

Los catálogos de modelos **caducan**. Por eso en este seminario no escribimos un nombre de modelo a mano en cuarenta celdas: lo elegimos **una vez**, comprobando que responde, y todo lo demás lo hereda.

0.1 ¿Qué tengo instalado?

Lo primero es saber sobre qué estamos construyendo. Estas son las versiones que usa el seminario; si las tuyas son muy distintas, algunas rutas de import habrán cambiado.

```python
import platform
import sys
import importlib.metadata as md

print("Python:", sys.version.split()[0], "|", platform.system(), platform.machine())
print()
PAQUETES = [
    "langgraph", "langgraph-prebuilt", "langgraph-checkpoint",
    "langchain", "langchain-core", "langchain-openai",
    "openai", "pydantic",
]
for paquete in PAQUETES:
    try:
        print(f"  {paquete:22} {md.version(paquete)}")
    except md.PackageNotFoundError:
        print(f"  {paquete:22} (NO instalado)")

import comun
print()
print("Daemon de Ollama :", comun.OLLAMA)
print("Endpoint OpenAI  :", comun.BASE_URL_OPENAI)
```

0.2 ¿Qué modelos tengo?

Esta lista es lo que Ollama **dice** que tienes. Tómalo como una pista, no como una garantía: en el paso siguiente comprobamos cuáles responden de verdad.

```python
from comun import modelos_listados

listados = modelos_listados()
print(f"Tienes {len(listados)} modelos:\n")
for nombre in listados:
    etiqueta = "nube" if "cloud" in nombre else "local"
    print(f"  [{etiqueta:5}] {nombre}")
```

0.3 La prueba de fuego: ¿cuál responde?

Aquí está la lección más importante de la libreta. Vamos a **enviarle un mensaje a cada modelo** y a quedarnos con el motivo exacto del fallo cuando lo haya. Son los errores reales de esta máquina, con su texto literal.

Sondear un modelo local la primera vez tarda: hay que cargarlo en memoria. Es normal.

```python
from comun import probar_modelo

# Sondeamos los de nube (fallan rápido) y el candidato local.
a_probar = list(dict.fromkeys([m for m in listados if "cloud" in m] + ["gemma4:e4b"]))

print(f"{'modelo':26} {'¿responde?':11} detalle")
print("-" * 104)
sondeos = {}
for nombre in a_probar:
    ok, detalle = probar_modelo(nombre)
    sondeos[nombre] = (ok, detalle)
    print(f"{nombre:26} {('SÍ' if ok else 'NO'):11} {detalle[:64]}")
```

No todos los fallos son iguales

El mensaje de error es la información más valiosa de todo el preflight, porque **cada causa exige una decisión distinta**.

```python
retirados = [m for m, (ok, detalle) in sondeos.items() if "retired" in detalle]
sin_cuota = [m for m, (ok, detalle) in sondeos.items() if "free usage" in detalle]
sanos = [m for m, (ok, _) in sondeos.items() if ok]

print(f"Modelos que RESPONDEN              : {len(sanos)}")
print(f"Modelos RETIRADOS por el proveedor : {len(retirados)}   {retirados}")
print(f"Modelos fuera del plan gratuito    : {len(sin_cuota)}")
print()
print("Tres finales distintos, tres causas distintas:")
print("  retirado  -> el proveedor lo quitó: hay que cambiar de modelo, no pagar")
print("  sin cuota -> existe y funciona, pero no entra en tu plan: es dinero")
print("  sano      -> adelante")
```

0.4 Elegir el modelo del seminario

`comun.elegir_modelo()` recorre los candidatos en orden de preferencia y devuelve el primero que **contesta de verdad**:

1. lo que digas en la variable de entorno `SEMINARIO_MODELO` (tu elección manda),
2. `gemma4:31b-cloud` — nube gratuita,
3. `gemma4:e4b` — local, sin necesidad de internet.

Fíjate en el detalle: **no basta con estar en la lista**. Se comprueba respondiendo.

```python
from comun import elegir_modelo, es_nube

MODELO = elegir_modelo()
print()
```

```
print("Modelo del seminario:", MODELO)
print("¿Vive en la nube?    ", es_nube(MODELO),
      "-> si es False, todo funciona sin conexión")
```

0.5 ¿Responde bien a una petición normal?

Comprobación trivial de que la tubería completa funciona: LangChain → endpoint compatible con OpenAI → daemon de Ollama → modelo (en la nube o en disco).

```
import comun

modelo = comun.llm()
respuesta = modelo.invoke("Responde únicamente con la palabra: listo")
print("respuesta del modelo:", respuesta.content.strip()[:120])
```

0.6 La prueba crítica: ¿sabe usar herramientas?

Todo el seminario se apoya en esto. Un agente es, en el fondo, un modelo que puede pedir que ejecutemos funciones. Si el modelo no es capaz de emitir `tool_calls`, ningún grafo que escribamos servirá de nada.

Le damos una herramienta y le pedimos explícitamente que la use. Si no la pide, paramos. Esta es la diferencia entre enterarse en el minuto 5 o descubrirlo en el minuto 150.

```
from langchain.tools import tool
from comun import seccion

@tool
def buscar_precio(producto: str) -> float:
    """Devuelve el precio estimado de un producto en EUR."""
    return 5.0

seccion("PRUEBA CRÍTICA: ¿sabe este modelo usar herramientas?")

respuesta = comun.llm().bind_tools([buscar_precio]).invoke(
    "¿Cuánto cuesta el pollo? Usa la herramienta buscar_precio."
)

print("tool_calls solicitados:", respuesta.tool_calls or "(ninguno)")
print()

assert respuesta.tool_calls, (
    "Este modelo NO ha pedido ninguna herramienta.\n"
    "Sin tool calling no hay agentes: cambia de modelo antes de seguir.\n"
    "Prueba con:  ollama pull gemma4:e4b\n"
    "y luego:     SEMINARIO_MODELO=gemma4:e4b  (o edita comun.CANDIDATOS)"
)
print("Tool calling confirmado. Todo listo para empezar.")
```

Si algo ha fallado

El daemon no responde (`No puedo hablar con Ollama`). Arranca el servicio:

```
ollama serve          # en otra terminal
```

Ningún modelo contesta. Descarga uno local, que además es gratis y sin cuota:

```
ollama pull gemma4:e4b
```

Los modelos de nube dan error de cuota o de modelo retirado. Es lo normal y no es culpa tuya: el plan gratuito cambia. Dos salidas:

- usar el modelo local `gemma4:e4b` (hace tool calling, verificado),
- o crear una API key en `https://ollama.com/settings/keys` y exportarla:

```
export OLLAMA_API_KEY="tu_clave"
```

Quiero forzar un modelo concreto sin tocar código:

```
SEMINARIO_MODELO=gemma4:e4b jupyter lab
```

Las peticiones se quedan esperando o dan `429`. El plan gratuito permite **una sola petición concurrente** (y una cuota mensual pequeña). Dos consecuencias para el taller:

- lanza los ejemplos **en secuencia**, no en paralelo: nada de *fan-out* entre varios agentes a la vez;
- si se agota la cuota, el retroceso de verdad es **local** (`gemma4:e4b`), no otro modelo de nube: la cuota es de la cuenta, no del modelo.

Recuerda también que los términos de Ollama piden ser mayor de edad y **una cuenta por persona**: cada alumno usa su propio *signin*, no una clave compartida de clase.

Lo que hemos aprendido y seguiremos usando

Idea	Por qué importa
Un modelo listado no es un modelo que funciona	Los catálogos caducan y las cuotas cambian
Hay que comprobar **respondiendo**, no leyendo una lista	Es la única prueba fiable
El tool calling es el requisito duro	Sin él no hay agentes, solo chat
El modelo se elige en un sitio	Cambiar de modelo no debe tocar 40 celdas

01 · El rol: quién es el agente

El mapa del seminario

Vamos a construir cuatro cosas. En un framework de «cadenas» viven escondidas dentro de una caja negra; en LangGraph son **objetos que se ven dibujados en el grafo**:

Concepto	Dónde vive en LangGraph	Cómo se ve
Rol	el prompt de sistema de un nodo	la función que escribes
Herramientas	`ToolNode` + una arista condicional	un nodo y una flecha
Memoria	`checkpointer` + `thread_id` (corto plazo) · `Store` (largo plazo)	lo que pasas en `config`
Planificación	las aristas agente y herramientas	el grafo que dibujas

Empezamos por el rol, porque es lo que convierte una llamada a un LLM en un *agente*: alguien con identidad, con un objetivo y con límites.

1.1 El grafo más pequeño posible

Antes de hablar de roles hay que ver qué es un grafo en LangGraph. Son tres piezas:

- **Estado** (`MessagesState`): el sobre que viaja entre nodos. Aquí, la lista de mensajes.
- **Nodo**: una función que recibe el estado y devuelve **solo lo que cambia**.
- **Aristas**: quién va después de quién.

Ese detalle del nodo es importante y sorprende al principio: un nodo no devuelve el estado completo, devuelve el parche. LangGraph lo fusiona con el estado anterior.

```python
from langgraph.graph import StateGraph, START, END, MessagesState
import comun

modelo = comun.llm()

def nodo_llm(estado: MessagesState):
    # Recibe el estado entero, devuelve SOLO la clave que modifica.
    return {"messages": [modelo.invoke(estado["messages"])]}

constructor = StateGraph(MessagesState)
constructor.add_node("llm", nodo_llm)
constructor.add_edge(START, "llm")
constructor.add_edge("llm", END)
grafo = constructor.compile()

comun.ver_grafo(grafo, "Grafo 1: el más pequeño")
```

Ese texto es el grafo en notación Mermaid. Pégalo ahí y lo verás como diagrama. Esta es la ventaja didáctica de LangGraph: **la estructura es un dato**, no algo que hay que imaginar. Vamos a ejecutarlo.

```
resultado = grafo.invoke({"messages": [{"role": "user", "content": "¿Quién eres?"}]})
print(resultado["messages"][-1].content[:400])
```

1.2 El rol es un parámetro, no una constante

Fíjate en lo que acabamos de ver: el modelo no tenía rol, así que respondió como «un asistente cualquiera». Ahora viene la primera ventaja de LangGraph: el rol **no** es una cadena escondida dentro de una clase. Es un dato que pasas al construir el grafo.

Vamos a fabricar el mismo grafo con dos roles distintos y hacerles la misma pregunta.

```
from langchain_core.messages import SystemMessage

def crear_agente_con_rol(rol: str):
    """Devuelve un grafo cuyo único nodo encarna ese rol."""
    def nodo(estado: MessagesState):
        mensajes = [SystemMessage(content=rol)] + estado["messages"]
        return {"messages": [modelo.invoke(mensajes)]}

    c = StateGraph(MessagesState)
    c.add_node("agente", nodo)
    c.add_edge(START, "agente")
    c.add_edge("agente", END)
    return c.compile()

PREGUNTA = "Tengo pollo, arroz y espinacas. ¿Qué hago?"

ROLES = {
    "Chef creativo": (
        "Eres un chef mediterráneo. Propones platos sabrosos y rápidos. "
        "Hablas de cocina, de texturas y de trucos."
    ),
    "Nutricionista": (
        "Eres un nutricionista deportivo. Te importan las calorías y los macronutrientes. "
        "Das siempre cifras concretas."
    ),
}

for nombre, rol in ROLES.items():
    salida = crear_agente_con_rol(rol).invoke(
        {"messages": [{"role": "user", "content": PREGUNTA}]}
    )
    print("=" * 72)
    print(nombre.upper())
    print("=" * 72)
    print(salida["messages"][-1].content[:420])
    print()
```

1.3 El rol da contenido, pero no garantiza la forma

Un rol bien escrito orienta el tono, el enfoque y el vocabulario. Lo que **no** hace es obligar al modelo a devolver una estructura concreta.

Vamos a comprobarlo de la forma más honesta posible: pidiéndolo por favor y contando cuántas veces nos hace caso de tres intentos.

```
import json
from pydantic import BaseModel, Field

class Receta(BaseModel):
    titulo: str = Field(description="Nombre del plato")
    minutos: int = Field(description="Tiempo total de preparación, en minutos")
    ingredientes: list[str] = Field(description="Ingredientes necesarios")

esquema = json.dumps(Receta.model_json_schema(), ensure_ascii=False)
peticion = (
```

```
    "Devuelve ÚNICAMENTE un objeto JSON válido, sin markdown ni texto adicional, "
    "que cumpla exactamente este JSON Schema:\n"
    f"{esquema}\n\n"
    "Receta rápida con pollo."
)
INSTANCIA = "Eres un chef. Cuando se te pide un formato, lo obedeces al pie de la letra."

validos = 0
for intento in range(1, 4):
    crudo = modelo.invoke([("system", INSTANCIA), ("human", peticion)]).content
    try:
        Receta.model_validate_json(crudo)
        validos += 1
        print(f"intento {intento}: JSON válido")
    except Exception as error:
        print(f"intento {intento}: NO válido ({type(error).__name__})")
        if intento == 1:
            print("   esto es lo que devolvió de verdad:")
            print("   " + crudo[:200].replace("\n", "\n   "))

print()
print(f"RESULTADO: {validos} de 3 intentos devolvieron JSON válido.")
print("Pedir el formato «por favor» no es un contrato: es una sugerencia.")
```

1.4 La forma se impone por el canal de herramientas

Para conseguir una estructura garantizada hay que dejar de pedir y empezar a **obligar**. `create_agent` acepta `response_format=` con un modelo de Pydantic.

Por debajo no hay magia: LangChain construye una **herramienta** cuyo esquema es tu modelo de Pydantic y le exige al modelo rellenarla. Es exactamente el mismo mecanismo que estudiaremos en el módulo de herramientas. Ese es el motivo de que este módulo vaya antes: el rol te da el *qué*, y el canal de herramientas te da el *cómo*, con garantías.

```
from langchain.agents import create_agent

agente = create_agent(
    modelo,
    tools=[],                       # todavía no hay herramientas propias
    system_prompt="Eres un chef mediterráneo. Propones platos rápidos y sabrosos.",
    response_format=Receta,         # <- el contrato de formato
)

salida = agente.invoke({"messages": [{"role": "user", "content": "Receta rápida con pollo."}]})

print("claves del estado devuelto:", list(salida.keys()))
receta = salida["structured_response"]
print("tipo devuelto:", type(receta).__name__)
print()
print("titulo      :", receta.titulo)
print("minutos     :", receta.minutos)
print("ingredientes:", receta.ingredientes)
```

1.5 Un rol que se puede verificar

Un rol serio no es solo una personalidad: son **restricciones comprobables**. Aquí le exigimos al agente que ninguna receta pase de 15 minutos, y lo comprobamos con Python.

La diferencia es enorme. Si el rol solo vive en el prompt, un incumplimiento lo descubre el usuario. Si además lo verificamos, lo descubrimos nosotros.

```
LIMITE = 15

agente_express = create_agent(
    modelo,
```

```
    tools=[],
    system_prompt=(
        "Eres un chef especializado en cocina exprés. "
        f"Ninguna receta puede superar los {LIMITE} minutos de preparación total."
    ),
    response_format=Receta,
)

receta = agente_express.invoke(
    {"messages": [{"role": "user", "content": "Propón algo con pollo y arroz."}]}
)["structured_response"]

print(f"Propuesta: {receta.titulo}  ->  {receta.minutos} min")
if receta.minutos <= LIMITE:
    print(f"Cumple el rol (<= {LIMITE} minutos).")
else:
    print(f"INCUMPLE el rol: {receta.minutos} > {LIMITE} minutos.")
    print("El prompt no bastaba; esto es lo que hay que verificar en código.")
```

Resumen del módulo

Idea	Detalle
Un nodo devuelve **el parche**, no el estado entero	LangGraph fusiona por ti
El grafo es un dato que se puede imprimir	`app.get_graph().draw_mermaid()`
El rol es un **parámetro** del grafo	Se cambia sin reescribir la lógica
El rol gobierna el contenido, no la forma	Pedir un formato es una sugerencia
La forma se impone por el canal de herramientas	`response_format=MiModelo`
Un rol maduro es una **restricción verificable**	Compruébala con Python

Ejercicios

1. **Tres roles, una pregunta.** Añade un tercer rol («abuelo gallego que cocina de aprovechamiento») y compara. ¿Cambia solo el tono o también la propuesta?

2. **Rol con dos límites.** Añade a `Receta` un campo `dificultad` con valores permitidos (`"facil"`, `"media"`, `"dificil"`) usando `Literal`. Ejecútalo cinco veces seguidas: ¿respeta siempre el rol de cocina exprés?

3. **Detecta el incumplimiento.** Cambia el rol para que *ninguna* receta pase de 5 minutos. Con un límite tan agresivo, ¿cuántas de 5 intentos incumplen? Escribe un bucle que lo cuente y muestre el porcentaje.

4. **El rol mínimo.** Ve bajando el `system_prompt` hasta dejarlo casi vacío. ¿En qué momento deja de comportarse como un chef? Esto enseña cuánto trabajo hace el prompt.

5. **Añade memoria de rol.** ¿Qué harías para que el agente recordara que eres alérgico al cilantro entre dos ejecuciones distintas? Apunta tu idea; la resolvemos en el módulo 03.

02 · Herramientas: el agente deja de inventar

En el módulo anterior vimos que la forma se impone por el **canal de herramientas**, pero sin explicar qué es una herramienta. Vamos a ello.

Una **herramienta** es una función de Python con dos añadidos que el modelo puede leer:

- un **nombre**,
- un **docstring**, que hace de descripción para el modelo.

Y nada más. El modelo no ejecuta nada: **pide** que se ejecute, y el grafo lo ejecuta. Esta separación es la idea central del módulo:

> El modelo **decide**. El grafo **ejecuta**. Tú **controlas qué puede decidir**.

2.1 Definir herramientas y ver lo que ve el modelo

El decorador `@tool` construye el esquema a partir de la firma y del docstring. Merece la pena mirar ese esquema: es literalmente el texto que el modelo recibe para decidir si usa la herramienta.

```python
import json
from langchain.tools import tool

@tool
def buscar_precio(producto: str) -> float:
    """Devuelve el precio estimado de un producto en EUR. Productos disponibles: pollo, arroz, espinacas."""
    precios = {"pollo": 5.0, "arroz": 2.0, "espinacas": 3.0}
    return precios.get(producto.lower(), 9.99)

@tool
def convertir_moneda(cantidad: float, origen: str, destino: str) -> float:
    """Convierte una cantidad de dinero entre EUR y USD."""
    tasas = {("EUR", "USD"): 1.10, ("USD", "EUR"): 0.90}
    return round(cantidad * tasas.get((origen.upper(), destino.upper()), 1.0), 2)

HERRAMIENTAS = [buscar_precio, convertir_moneda]

print("Estas herramientas son objetos Python normales. Puedo llamarlas yo mismo:")
print("  buscar_precio ->", buscar_precio.invoke({"producto": "pollo"}))
print()
print("Y este es el contrato que recibe el modelo:")
print(json.dumps(buscar_precio.tool_call_schema.model_json_schema(), ensure_ascii=False, indent=2))
```

Ese JSON es todo lo que el modelo sabe de nuestra función. De ahí se deduce una regla práctica: **un docstring vago hace una herramienta inservible**. Si no describes bien cuándo usarla, el modelo no la usará, o la usará mal.

2.2 El bucle ReAct, escrito a mano

Ahora montamos el ciclo completo. Es un grafo con dos nodos:

```
START -> agente -> (¿pidió herramientas?) -> herramientas -> agente -> ... -> END
```

- **`agente`**: le pregunta al modelo qué hacer. Si quiere una herramienta, lo dice.
- **`herramientas`**: ejecuta lo que pidió y le devuelve el resultado.

La arista del medio es **condicional**: es la decisión de si el agente sigue trabajando o ya ha terminado. Ahí está toda la «inteligencia» del bucle.

Un detalle que hace perder veinte minutos si no lo sabes: `tools_condition` devuelve por defecto los literales `"tools"` y `"__end__"`. Si tu nodo se llama de otra forma —como aquí, `herramientas`— hay que **mapear** la salida. LangGraph no adivina por parecido de nombres. Eso es exactamente lo que hace el tercer argumento de `add_conditional_edges`.

```python
from langgraph.graph import StateGraph, START, END, MessagesState
from langgraph.prebuilt import ToolNode, tools_condition
import comun

modelo = comun.llm()
modelo_con_herramientas = modelo.bind_tools(HERRAMIENTAS)

def nodo_agente(estado: MessagesState):
    """El modelo decide qué hacer con la conversación."""
    return {"messages": [modelo_con_herramientas.invoke(estado["messages"])]}

constructor = StateGraph(MessagesState)
constructor.add_node("agente", nodo_agente)
constructor.add_node("herramientas", ToolNode(HERRAMIENTAS))
constructor.add_edge(START, "agente")

# La arista condicional. El mapa traduce la salida de tools_condition
# ("tools" / "__end__") al nombre real de nuestro nodo.
constructor.add_conditional_edges(
    "agente", tools_condition, {"tools": "herramientas", "__end__": END}
)

constructor.add_edge("herramientas", "agente")   # <- aquí está el bucle
agente_manual = constructor.compile()

comun.ver_grafo(agente_manual, "Grafo 2: el bucle ReAct")
```

Fíjate en la flecha que vuelve de `herramientas` a `agente`. **Esa flecha es el bucle.** En un framework de cadenas esto está escondido dentro de un `AgentExecutor`; aquí lo estás viendo dibujado, y si quisieras poner un límite de iteraciones o un humano de por medio, sabrías exactamente dónde pincharlo.

2.3 Ejecutarlo y, sobre todo, leer la traza

Lo importante no es que responda bien, sino **ver cómo lo ha hecho**. La lista de mensajes del estado es la caja de cristal del agente:

Tipo de mensaje	Quién lo emite	Qué significa
`HumanMessage`	tú	la petición
`AIMessage` con `tool_calls`	el modelo	«quiero ejecutar esto»
`ToolMessage`	tu código	el resultado de la herramienta
`AIMessage` final	el modelo	la respuesta ya con los datos reales

Si no ves `tool_calls`, el modelo **se lo está inventando**. Ese es el síntoma exacto que hay que saber detectar.

```python
import comun

entrada = "¿Cuánto cuesta el pollo? ¿Y cuánto son 100 EUR en USD?"
resultado = agente_manual.invoke({"messages": [{"role": "user", "content": entrada}]})

print("PREGUNTA:", entrada)
print()
```

```
comun.ver_mensajes(resultado)
print()
print("RESPUESTA FINAL:")
print(resultado["messages"][-1].content[:300])
```

Lee la traza en voz alta conmigo, porque es el corazón del módulo:

1. `[00] HumanMessage` — la pregunta.
2. `[01] AIMessage -> TOOL_CALLS: buscar_precio(...), convertir_moneda(...)` — el modelo **no contesta**: pide dos herramientas. Y las pide **a la vez**, en paralelo.
3. `[02]` y `[03] ToolMessage` — los resultados reales: `5.0` y `110.0`. Esos números no los ha inventado el modelo: los ha calculado Python.
4. `[04] AIMessage` — ahora sí responde, con los datos que le hemos dado.

Esa es la diferencia entre un chatbot que opina y un agente que consulta.

2.4 Cuando una herramienta falla

Las herramientas son código, y el código falla. Una división por cero, una API caída, un producto que no existe. Si el error se propaga, el agente se cae entero.

`ToolNode` puede convertir la excepción en un `ToolMessage` de error, de modo que el modelo **se entere y se recupere**. Compara las dos situaciones.

```
from langchain.tools import tool

@tool
def dividir(a: float, b: float) -> float:
    """Divide el número a entre el número b."""
    return a / b

# 1) La herramienta, invocada directamente: el error sale disparado.
try:
    dividir.invoke({"a": 6, "b": 0})
except Exception as error:
    print(f"Sin protección -> {type(error).__name__}: {error}")
```

```
from langgraph.graph import StateGraph, START, END, MessagesState
from langgraph.prebuilt import ToolNode, tools_condition
from langchain_core.messages import SystemMessage
import comun

# 2) El mismo fallo, pero dentro de un grafo que sí maneja el error.
modelo_div = modelo.bind_tools([dividir])

def nodo_div(estado: MessagesState):
    sistema = SystemMessage(content=(
        "Usa la herramienta dividir. Si la herramienta falla, "
        "explica el error al usuario en una frase."
    ))
    return {"messages": [modelo_div.invoke([sistema] + estado["messages"])]}

c = StateGraph(MessagesState)
c.add_node("agente", nodo_div)
c.add_node("herramientas", ToolNode([dividir], handle_tool_errors=True))
c.add_edge(START, "agente")
c.add_conditional_edges("agente", tools_condition, {"tools": "herramientas", "__end__": END})
c.add_edge("herramientas", "agente")
grafo_div = c.compile()

salida = grafo_div.invoke({"messages": [{"role": "user", "content": "Divide 6 entre 0."}]})
comun.ver_mensajes(salida)
print()
print("RESPUESTA FINAL:")
print(salida["messages"][-1].content[:300])
```

Fíjate en la `ToolMessage`: contiene **el texto del error**, no una excepción. Eso es `handle_tool_errors=True` en acción. La consecuencia práctica es enorme: el agente puede **leer** que su herramienta ha fallado y reaccionar —pedir otro dato, disculparse, proponer una alternativa— en lugar de morir con un `Traceback`.

La lección de diseño: los errores de tus herramientas son **información para el agente**, no solo un problema para tu `try/except`.

2.5 Todo esto, en una línea: `create_agent`

Acabamos de escribir a mano el bucle que implementan todos los agentes. Ahora la pregunta honesta: ¿para qué tanto trabajo si existe un atajo?

Porque **ahora sabes qué hace el atajo por dentro**. Y eso importa el día que el agente se comporte raro: sabrás si mirar el prompt, la arista condicional o el manejo de errores.

`create_agent` construye exactamente el mismo grafo que hemos montado nosotros.

```python
from langchain.agents import create_agent

agente_atajo = create_agent(
    modelo,
    tools=HERRAMIENTAS,
    system_prompt="Eres un chef conciso. Usa las herramientas cuando necesites datos.",
)

comun.ver_grafo(agente_atajo, "Grafo 2b: lo que construye create_agent por dentro")
```

```python
resultado = agente_atajo.invoke(
    {"messages": [{"role": "user", "content": "¿Cuánto cuesta el arroz y a cuánto van 50 USD?"}]}
)
comun.ver_mensajes(resultado)
print()
print("RESPUESTA:", resultado["messages"][-1].content[:300])
```

Ese grafo tiene más nodos que el nuestro (gestión de errores de herramientas, de modelo, etc.), pero la estructura es la misma que has construido a mano. **Ya no es una caja negra.**

Resumen del módulo

Idea	Detalle
Una herramienta es una función + nombre + docstring	El docstring es el prompt de la herramienta
El modelo **no ejecuta**: pide	`AIMessage.tool_calls` es la petición
El grafo ejecuta y devuelve `ToolMessage`	Ahí llegan los datos reales
La arista condicional es el bucle	`tools_condition` decide seguir o parar
`tools_condition` devuelve `"tools"` / `"__end__"`	Si tu nodo se llama distinto, **mapea**
Sin `tool_calls` en la traza, el modelo se lo inventó	Es el síntoma a detectar
`handle_tool_errors=True` convierte el fallo en observación	El agente se recupera solo
`create_agent` hace lo mismo en una línea	Pero ahora sabes qué hay dentro

Ejercicios

1. **Herramienta que miente.** Haz que `buscar_precio` devuelva siempre `0.01`. ¿Se da cuenta el agente de que el precio es absurdo? Comenta el resultado: el agente confía en sus herramientas, y eso es una responsabilidad tuya, no del modelo.

2. **Docstring vago.** Cambia el docstring de `buscar_precio` por `"Busca cosas."` y repite la pregunta del apartado 2.3. ¿Sigue usándola correctamente? Escribe la diferencia.

3. **Tres herramientas en paralelo.** Añade `contar_calorias(ingrediente)`. Pregunta por precio, moneda y calorías en la misma frase. ¿Cuántos `TOOL_CALLS` ves en un solo `AIMessage`?

4. **Una herramienta que siempre falla.** Añade una que lance `ValueError("servicio caído")` y pregunta algo que la requiera. Describe cómo reacciona el modelo con `handle_tool_errors=True` y qué pasa con `handle_tool_errors=False`.

5. **Poner un límite.** Un bucle ReAct puede girar indefinidamente. Busca `recursion_limit` en la documentación de LangGraph y limita el grafo a 4 pasos. ¿Qué excepción lanza al pasarse?

6. **Tu propia herramienta.** Escribe una que consulte algo real (la hora del sistema, una API pública, un fichero local). Cuida el docstring: es lo único que el modelo verá.

03 · Memoria: continuidad y aprendizaje

Un LLM no recuerda nada. Cada llamada es un mundo nuevo, sin pasado. Cuando un chatbot «se acuerda» de tu nombre, lo que ocurre en realidad es que **alguien le está reenviando la conversación entera** en cada turno.

En LangGraph ese «alguien» tiene nombre y apellidos, y hay que distinguir **dos memorias que se confunden constantemente**:

	Memoria a corto plazo	Memoria a largo plazo
Qué guarda	la conversación en curso	hechos sobre el usuario
Dónde vive	`checkpointer`	`Store`
Se identifica por	`thread_id` (hilo)	*namespace* (usuario)
Sobrevive a	los turnos del hilo	los hilos **y** los procesos*
Analogía	lo que recuerdas de esta charla	lo que sabes de tu amigo

* si usas un `Store` persistente. El de esta libreta vive en memoria y muere con el proceso; lo comentamos al final.

3.1 Sin checkpointer, el agente tiene amnesia

Empecemos por ver el problema, no la solución. Este grafo no tiene memoria: cada `invoke` es una conversación nueva.

```python
from langgraph.graph import StateGraph, START, END, MessagesState
from langchain_core.messages import SystemMessage
import comun

modelo = comun.llm()
PERSONALIDAD = "Eres un chef conciso y cercano."

def crear_grafo(checkpointer=None):
    """El mismo grafo, con o sin memoria, según lo que le pasemos."""
    def nodo(estado: MessagesState):
        mensajes = [SystemMessage(content=PERSONALIDAD)] + estado["messages"]
        return {"messages": [modelo.invoke(mensajes)]}

    c = StateGraph(MessagesState)
    c.add_node("agente", nodo)
    c.add_edge(START, "agente")
    c.add_edge("agente", END)
    return c.compile(checkpointer=checkpointer)

sin_memoria = crear_grafo()

print("Turno 1")
r1 = sin_memoria.invoke({"messages": [{"role": "user", "content": "Me llamo Luis y odio el cilantro."}]})
print("  ", r1["messages"][-1].content[:150])

print("\nTurno 2 (en teoría, la misma conversación)")
r2 = sin_memoria.invoke({"messages": [{"role": "user", "content": "¿Cómo me llamo?"}]})
print("  ", r2["messages"][-1].content[:200])

print(f"\nMensajes que ha visto el modelo en el turno 2: {len(r2['messages'])}")
print("El historial del turno 1 se ha tirado a la basura.")
```

3.2 Con checkpointer, hay hilo

Añadimos `InMemorySaver` al compilar y pasamos un `thread_id` en cada llamada. **Ese identificador es la conversación.** Cambia el `thread_id` y cambias de interlocutor.

```python
from langgraph.checkpoint.memory import InMemorySaver

con_memoria = crear_grafo(InMemorySaver())
cfg_A = {"configurable": {"thread_id": "alumno-A"}}

print("Turno 1")
r1 = con_memoria.invoke({"messages": [{"role": "user", "content": "Me llamo Luis y odio el cilantro."}]}, config=
    cfg_A)
print("  ", r1["messages"][-1].content[:150])

print("\nTurno 2")
r2 = con_memoria.invoke({"messages": [{"role": "user", "content": "¿Cómo me llamo? ¿Qué no me gusta?"}]}, config=
    cfg_A)
print("  ", r2["messages"][-1].content[:250])

print(f"\nMensajes que ha visto el modelo en el turno 2: {len(r2['messages'])}")
print("Aquí está la memoria: el historial crece y viaja en cada llamada.")
```

3.3 Hilos distintos, memorias distintas

Esto es lo que hace útil al `thread_id`: aislamiento gratis entre conversaciones. Dos alumnos, dos hilos, cero contaminación.

```python
cfg_B = {"configurable": {"thread_id": "alumno-B"}}

print("Preguntamos lo mismo en el hilo B, que nunca ha oído hablar de Luis:")
r = con_memoria.invoke({"messages": [{"role": "user", "content": "¿Cómo me llamo?"}]}, config=cfg_B)
print("  ", r["messages"][-1].content[:250])
```

3.4 ¿Dónde está la memoria exactamente?

No hay magia ni «estado oculto del modelo». Hay una lista de mensajes guardada en un `checkpoint`, y podemos abrirla. Merece la pena hacerlo una vez: desmitifica el concepto para siempre.

```python
estado = con_memoria.get_state(cfg_A)

print("Claves guardadas en el checkpoint:", list(estado.values.keys()))
print(f"Mensajes persistidos en el hilo alumno-A: {len(estado.values['messages'])}\n")
for i, mensaje in enumerate(estado.values["messages"]):
    texto = (mensaje.content or "").replace("\n", " ")[:70]
    print(f"  [{i}] {type(mensaje).__name__:18} {texto}")

print()
print("La memoria es esto: una lista que se guarda y se vuelve a cargar.")
```

3.5 Memoria a largo plazo: el `Store`

El `checkpointer` resuelve «esta conversación». Pero hay cosas que no pertenecen a un hilo concreto: que eres alérgico a los frutos secos, que prefieres recetas económicas, que cocinas para cuatro. Eso debe sobrevivir a **cualquier** conversación.

Para eso está el `Store`: un almacén de clave-valor organizado por *namespaces*, pensado para datos del usuario. Y aquí está la diferencia clave frente a meterlo en el prompt:

- el `Store` **no ocupa contexto** hasta que hace falta,
- se consulta **desde dentro de una herramienta**, cuando el agente lo necesita,

- y por tanto no se pierde al recortar el historial.

Dos herramientas: una que guarda y otra que lee. Fíjate en `runtime.store`.

```python
from langgraph.store.memory import InMemoryStore
from langchain.agents import create_agent
from langchain.tools import tool, ToolRuntime

ALUMNO = "luis"           # el namespace identifica al usuario, no al hilo
almacen = InMemoryStore()

@tool
def guardar_preferencia(clave: str, valor: str, runtime: ToolRuntime) -> str:
    """Guarda un dato del alumno (una alergia, una preferencia) para futuras conversaciones."""
    item = runtime.store.get((ALUMNO,), "preferencias")
    datos = dict(item.value) if item else {}
    datos[clave] = valor
    runtime.store.put((ALUMNO,), "preferencias", datos)
    return f"guardado: {clave} = {valor}"

@tool
def leer_preferencias(runtime: ToolRuntime) -> str:
    """Devuelve todos los datos guardados del alumno."""
    item = runtime.store.get((ALUMNO,), "preferencias")
    return str(item.value) if item else "no hay nada guardado"

chef = create_agent(
    modelo,
    tools=[guardar_preferencia, leer_preferencias],
    store=almacen,                        # <- aquí conectamos el almacén
    system_prompt=(
        "Eres un chef conciso. Cuando el alumno te cuente un dato personal "
        "relevante para cocinar, guárdalo con guardar_preferencia. "
        "Antes de proponer un plato, consulta leer_preferencias."
    ),
)

print("Almacén creado. Estado inicial:", almacen.get((ALUMNO,), "preferencias"))
```

3.6 La prueba que lo demuestra: memoria que cruza hilos

Guardamos en el **hilo 1** y preguntamos en el **hilo 2**, que es una conversación completamente nueva. El `checkpointer` no puede ayudar aquí: los hilos no se hablan. Si el agente lo recuerda, es porque lo ha leído del `Store`.

```python
# Hilo 1: el alumno cuenta una alergia.
r1 = chef.invoke(
    {"messages": [{"role": "user", "content": "Soy alérgico a los frutos secos. Apúntalo."}]},
    config={"configurable": {"thread_id": "hilo-1"}},
)
print("HILO 1:", r1["messages"][-1].content[:200])
print("Almacén tras el hilo 1:", almacen.get((ALUMNO,), "preferencias").value)
print()

# Hilo 2: conversación nueva. ¿Se acuerda?
r2 = chef.invoke(
    {"messages": [{"role": "user", "content": "¿Qué alergias tengo? Propón una cena."}]},
    config={"configurable": {"thread_id": "hilo-2"}},
)
print("HILO 2:", r2["messages"][-1].content[:400])
print()
comun.ver_mensajes(r2)
```

Y ahí está la distinción completa:

- El **hilo 2** no sabe nada del hilo 1 (`checkpointer`).

- Pero el agente **sí** conoce la alergia, porque la ha leído del `Store`.
- En la traza verás la `ToolMessage` con el contenido del almacén: el dato entró en la conversación **en el momento en que hacía falta**, y no antes.

3.7 Tres avisos que ahorran disgustos

1. Olvidar el `checkpointer` no da error: no hace nada. Es el fallo silencioso más frecuente. Si tu agente parece no recordar, lo primero que hay que mirar es si compilaste con `checkpointer=` y si estás pasando el mismo `thread_id`. Un `thread_id` nuevo es, por definición, una conversación nueva.

2. La memoria en RAM muere con el proceso. `InMemorySaver` e `InMemoryStore` viven en memoria: al reiniciar el cuaderno, todo desaparece. Para que sobreviva necesitas un almacén persistente, por ejemplo SQLite:

```
from langgraph.checkpoint.sqlite import SqliteSaver
## se instala aparte:  pip install langgraph-checkpoint-sqlite
with SqliteSaver.from_conn_string("memoria.db") as checkpointer:
    grafo = crear_grafo(checkpointer)
```

3. El historial crece sin límite. El turno 40 arrastra los 39 anteriores, y eso se paga en tokens, en dinero y en latencia. Las soluciones habituales son **recortar** los mensajes antiguos o **resumirlos**; LangChain 1.x trae middleware para ambas (`SummarizationMiddleware`, `TrimMessagesMiddleware`). No lo usamos aquí para no esconder la mecánica, pero es lo siguiente que hay que aprender.

Resumen del módulo

Idea	Detalle
Un LLM no recuerda: le reenvías el historial	La memoria es un dato, no una facultad
`checkpointer` + `thread_id` = memoria de conversación	Mismo id, misma conversación
Hilos distintos están aislados por construcción	Útil para multiusuario
La memoria es una lista de mensajes que se puede inspeccionar	`get_state(config)`
`Store` = memoria de largo plazo por usuario	Namespaces, no hilos
El `Store` se consulta desde herramientas y no ocupa contexto	Solo entra lo que se necesita
Sin `checkpointer`, la memoria falla **en silencio**	El error más caro del módulo

Ejercicios

1. **El mismo hilo, otro nombre.** En el apartado 3.2, pregunta «¿Cómo me llamo?» usando el `thread_id .ªlumno-B"`. ¿Por qué no lo sabe, si el grafo tiene memoria?

2. **Persistencia real.** Instala `langgraph-checkpoint-sqlite`, cambia `InMemorySaver` por `SqliteSaver` y reinicia el kernel. ¿Sobrevive la conversación de `alumno-A`? ¿Y el `InMemoryStore`? ¿Qué habría que cambiar para que el `Store` también persista?

3. **Namespace por alumno.** Modifica el `Store` para que el namespace sea `(.ªlumnos", nombre)` en lugar de una constante. Hazlo con `context_schema` para no escribir el nombre a fuego. Después guarda datos de dos alumnos y comprueba que no se mezclan.

4. **El coste de la memoria.** Activa la memoria y mide cuántos mensajes viajan en el turno 5, el 10 y el 20. ¿Crece linealmente? ¿Qué pasa con la latencia? Escribe los números.

5. **Olvido selectivo.** Añade una herramienta `olvidar(clave)` que borre una preferencia del `Store`. Pídele al agente que olvide la alergia y comprueba que deja de aplicarla.

6. **Discusión para clase.** Una memoria que se equivoca es peor que no tener memoria: ¿qué pasa si el `Store` guarda «soy alérgico a los frutos secos» y el usuario luego dice «ya no lo soy»? ¿Quién decide cuándo el dato caduca? Diseña tu regla.

04 · Planificación: decidir antes de actuar

En el módulo 02 construimos un bucle **ReAct**. Merece la pena parar y ver qué es exactamente lo que hace, porque tiene una limitación estructural:

> ReAct decide **un paso cada vez**, mirando solo lo que ya ha pasado.

Eso es *reactivo*, no *planificado*. El agente no tiene una visión de conjunto: va palpando. Para tareas cortas es perfecto. Para tareas largas aparece el problema clásico: se pierde, repite pasos o se queda en bucle.

Estrategia	Cómo decide	Cuándo brilla	Cuándo falla
ReAct	un paso, mirando el resultado anterior	tareas cortas, entornos cambiantes	tareas de muchos pasos
Plan-and-execute	todos los pasos primero, luego los ejecuta	tareas largas y previsibles	si el plan inicial está mal
Reflexión	actúa, se critica y corrige	calidad del resultado	cuesta más llamadas

Empecemos viendo el ReAct por dentro.

4.1 ReAct bajo el microscopio

Le pedimos algo que **obliga a encadenar**: primero hay que consultar un precio, y solo después se puede convertir a otra moneda. El segundo paso depende del primero, así que no se pueden hacer en paralelo.

```python
from langchain.tools import tool
from langgraph.graph import StateGraph, START, END, MessagesState
from langgraph.prebuilt import ToolNode, tools_condition
import comun

modelo = comun.llm()

@tool
def buscar_precio(producto: str) -> float:
    """Devuelve el precio de un producto en EUR por kilogramo."""
    return {"pollo": 5.0, "arroz": 2.0, "espinacas": 3.0}.get(producto.lower(), 9.99)

@tool
def convertir_moneda(cantidad: float, origen: str, destino: str) -> float:
    """Convierte una cantidad de dinero entre EUR y USD."""
    return round(cantidad * {("EUR", "USD"): 1.10, ("USD", "EUR"): 0.90}
                 .get((origen.upper(), destino.upper()), 1.0), 2)

HERRAMIENTAS = [buscar_precio, convertir_moneda]
modelo_ct = modelo.bind_tools(HERRAMIENTAS)

def nodo_agente(estado: MessagesState):
    return {"messages": [modelo_ct.invoke(estado["messages"])]}

c = StateGraph(MessagesState)
c.add_node("agente", nodo_agente)
```

```
c.add_node("herramientas", ToolNode(HERRAMIENTAS))
c.add_edge(START, "agente")
c.add_conditional_edges("agente", tools_condition, {"tools": "herramientas", "__end__": END})
c.add_edge("herramientas", "agente")
react = c.compile()
```

```
TAREA = "¿Cuánto cuestan 3 kg de pollo en USD? Necesito el precio en EUR primero."
resultado = react.invoke({"messages": [{"role": "user", "content": TAREA}]})

print("TAREA:", TAREA)
print()
comun.ver_mensajes(resultado)

vueltas = sum(1 for m in resultado["messages"] if getattr(m, "tool_calls", None))
print()
print(f"Vueltas del bucle ReAct: {vueltas}")
print("Cada vuelta es una llamada al modelo. El agente no sabía al empezar cuántas haría.")
```

Lo que acabas de ver es el bucle decidiendo **sobre la marcha**: en la primera vuelta solo podía pedir el precio; con ese dato en la mano, en la segunda ya pudo convertir. Nadie planificó nada de antemano.

Fíjate también en el coste: **cada vuelta es una llamada al modelo**. Si el plan de la tarea hubiera estado claro desde el principio, podríamos haber ahorrado alguna.

4.2 Plan-and-execute: la planificación es un campo más del estado

Aquí está la revelación de este módulo. Planificar no requiere ningún framework especial: requiere **un campo `plan` en el estado y un contador**. Eso es todo.

El grafo pasa a tener tres nodos:

```
START -> planificar -> ejecutar -> (¿quedan pasos?) -> ejecutar -> ... -> redactar -> END
```

- **planificar**: el modelo descompone la tarea en pasos. Usamos salida estructurada para obtener una lista, no un texto libre.
- **ejecutar**: coge el paso `estado["paso"]`, lo resuelve (con herramientas) y lo apunta.
- **redactar**: con todos los resultados, escribe el informe final.

La diferencia con ReAct no está en el LLM: está en **dónde vive el plan**.

```
import operator
from typing import Annotated, Literal
from typing_extensions import TypedDict
from pydantic import BaseModel, Field
from langchain.agents import create_agent
from langchain_core.messages import SystemMessage

class Plan(BaseModel):
    pasos: list[str] = Field(
        description="Entre 2 y 3 pasos concretos, ordenados y verificables. Máximo 3."
    )

class EstadoPlan(TypedDict):
    peticion: str
    plan: list[str]
    paso: int                                          # el contador del bucle
    resultados: Annotated[list[str], operator.add]     # se ACUMULAN
    informe: str

planificador = modelo.with_structured_output(Plan, method="function_calling")

# Un ejecutor con herramientas: resuelve cada paso por separado.
ejecutor = create_agent(
    modelo,
```

```
    tools=HERRAMIENTAS,
    system_prompt="Resuelves un único paso concreto. Sé breve y usa las herramientas.",
)

def nodo_planificar(estado: EstadoPlan):
    plan = planificador.invoke(
        "Descompón esta tarea en pasos. Tarea: " + estado["peticion"]
    )
    listado = "\n".join(f"{i+1}. {p}" for i, p in enumerate(plan.pasos))
    print("PLAN PROPUESTO:\n" + listado + "\n")
    return {"plan": plan.pasos, "paso": 0, "resultados": []}

def nodo_ejecutar(estado: EstadoPlan):
    actual = estado["plan"][estado["paso"]]
    print(f"--> ejecutando paso {estado['paso'] + 1}: {actual}")
    salida = ejecutor.invoke({"messages": [{"role": "user", "content": actual}]})
    return {
        "resultados": [f"Paso {estado['paso'] + 1} - {actual}\n{salida['messages'][-1].content}"],
        "paso": estado["paso"] + 1,
    }

def quedan_pasos(estado: EstadoPlan) -> Literal["seguir", "redactar"]:
    return "seguir" if estado["paso"] < len(estado["plan"]) else "redactar"

def nodo_redactar(estado: EstadoPlan):
    material = "\n\n".join(estado["resultados"])
    informe = modelo.invoke([
        SystemMessage(content="Redactas informes breves y claros a partir de material dado."),
        ("human", f"Redacta un informe final con este material:\n\n{material}"),
    ]).content
    return {"informe": informe}

g = StateGraph(EstadoPlan)
g.add_node("planificar", nodo_planificar)
g.add_node("ejecutar", nodo_ejecutar)
g.add_node("redactar", nodo_redactar)
g.add_edge(START, "planificar")
g.add_edge("planificar", "ejecutar")
g.add_conditional_edges("ejecutar", quedan_pasos, {"seguir": "ejecutar", "redactar": "redactar"})
g.add_edge("redactar", END)
grafo_plan = g.compile()

comun.ver_grafo(grafo_plan, "Grafo 4: planificar y ejecutar")
```

```
salida = grafo_plan.invoke(
    {"peticion": "¿Cuánto cuestan 3 kg de pollo en USD y cuánto ahorro si compro arroz en vez de espinacas?"},
    config={"recursion_limit": 30},
)

print("\n" + "=" * 70)
print("TRAZA DE RESULTADOS")
print("=" * 70)
for bloque in salida["resultados"]:
    print("-", bloque[:220].replace("\n", " "))
    print()
print("=" * 70)
print("INFORME FINAL")
print("=" * 70)
print(salida["informe"][:700])
```

Compara las dos ejecuciones:

	ReAct (4.1)	Plan-and-execute (4.2)
Llamadas al modelo	las que hagan falta, sin saber cuántas	1 para el plan + 1 por paso + 1 informe

	ReAct (4.1)	Plan-and-execute (4.2)
Visibilidad	ninguna hasta que termina	el plan es **legible** antes de ejecutar
Intervención	difícil	trivial: puedes revisar el plan
Coste si algo falla	se pierde el trabajo previo	se reejecuta un paso concreto

Esa penúltima fila es la clave práctica: **un plan explícito se puede revisar, aprobar o corregir antes de gastar dinero**. Que es exactamente lo que viene ahora.

4.3 Intervención humana: parar el grafo a mitad

Un agente autónomo que puede gastar dinero, enviar correos o borrar ficheros necesita un freno. LangGraph lo trae de serie: `interrupt()` **pausa el grafo** y lo deja esperando una decisión. Para reanudarlo, `Command(resume=...)`.

Requiere `checkpointer`: es lo que permite guardar el estado mientras el grafo está parado. Sin él, no hay nada que reanudar.

```python
from langgraph.types import interrupt, Command
from langgraph.checkpoint.memory import InMemorySaver
from langchain_core.messages import AIMessage

def nodo_compra(estado: MessagesState):
    propuesta = modelo.invoke([
        SystemMessage(content="Eres un chef conciso. Propones una lista de la compra."),
        ("human", "Propón la lista de la compra para una cena de 4 personas."),
    ]).content

    # AQUÍ se para el grafo. Lo que devuelve interrupt() es lo que
    # el humano decida al reanudar.
    decision = interrupt({
        "propuesta": propuesta,
        "pregunta": "¿Aprobamos este gasto?",
    })

    return {"messages": [AIMessage(content=f"DECISIÓN HUMANA: {decision}")]}

g2 = StateGraph(MessagesState)
g2.add_node("compra", nodo_compra)
g2.add_edge(START, "compra")
g2.add_edge("compra", END)
grafo_hitl = g2.compile(checkpointer=InMemorySaver())

cfg = {"configurable": {"thread_id": "compra-1"}}
print("Primera invocación: el grafo se detiene solo.")
parada = grafo_hitl.invoke(
    {"messages": [{"role": "user", "content": "Prepara la compra."}]}, config=cfg
)

interrupcion = parada["__interrupt__"][0].value
print("\nEl grafo está EN PAUSA y nos pide una decisión:")
print("  Pregunta :", interrupcion["pregunta"])
print("  Propuesta:", interrupcion["propuesta"][:220].replace("\n", " "))
```

```python
print("El humano responde. El grafo se reanuda EXACTAMENTE donde se quedó.\n")
final = grafo_hitl.invoke(Command(resume="Aprobado, pero sin cilantro."), config=cfg)

print(final["messages"][-1].content[:300])
print()
comun.ver_mensajes(final)
```

Ese patrón —**parar, preguntar, reanudar**— es la base de cualquier agente que se pueda poner en producción. Para agentes creados con `create_agent` existe una forma más declarativa:

```python
from langchain.agents.middleware import HumanInTheLoopMiddleware

agente = create_agent(
    modelo, tools=[enviar_correo],
    middleware=[HumanInTheLoopMiddleware(interrupt_on={"enviar_correo": True})],
    checkpointer=InMemorySaver(),
)
```

Cuatro reglas de `interrupt()` que hay que interiorizar

1. **El nodo se vuelve a ejecutar entero al reanudar.** Todo el código *anterior* a `interrupt()` se repite. Si antes lanzas una petición de red, la lanzarás dos veces: hazlo idempotente.
2. **Nunca lo metas en un `try/except` desnudo.** `interrupt()` pausa lanzando una excepción interna; si te la comes, el grafo no se detiene y no entiendes por qué.
3. **Nunca lo pongas dentro de un bucle `while`.** La correspondencia entre llamada y reanudación es **por posición**: un bucle hace que cada reanudación reproduzca varias interrupciones. Un `interrupt()` por nodo, y el bucle se hace con aristas condicionales.
4. **La pausa no sobrevive sin `checkpointer`.** Y hay que reanudar con el **mismo** `thread_id`.

Resumen del módulo

Idea	Detalle
ReAct decide un paso cada vez	Reactivo: no ve el conjunto
Planificar es tener un campo `plan` en el estado	No hace falta framework nuevo
El plan explícito se puede leer, revisar y aprobar	Es la ventaja real, no la elegancia
Un contador + arista condicional = bucle controlado	`estado["paso"]`
`interrupt()` pausa y `Command(resume=...)` reanuda	Requiere `checkpointer`
El nodo se reejecuta al reanudar	Los efectos secundarios deben ser idempotentes

El mapa completo del seminario

Concepto	Dónde vive	Lo has visto en
Rol	`system_prompt` de un nodo	Módulo 01
Herramientas	`ToolNode` + arista condicional	Módulo 02
Memoria	`checkpointer` + `thread_id`, `Store`	Módulo 03
Planificación	campo `plan` + contador en el estado	Módulo 04

Y `create_agent` empaqueta los tres primeros. La planificación es lo que **tú** añades encima, porque depende de tu problema: no hay una estrategia que gane siempre.

Ejercicios

1. **Menos llamadas.** En el apartado 4.2, haz que el plan tenga 1 solo paso para una tarea que ReAct resolvía en 2 vueltas. ¿Quién gasta menos llamadas al modelo? Mide cuántas hace

cada estrategia con un contador.

2. **Un paso que falla.** Cambia el ejecutor para que el paso 2 siempre falle (por ejemplo, pidiendo un producto que no existe). ¿Sigue el plan adelante? Diseña qué debería pasar: ¿reintentar, saltar o abortar? Impleméntalo con una arista condicional más.

3. **Revisar el plan antes de ejecutarlo.** Mete un `interrupt()` **entre** `planificar` y `ejecutar`, de modo que el humano vea el plan y pueda aprobarlo o sustituirlo. Este es el patrón real de los agentes que se usan en producción: `interrupt` con `response_schema` para validar la respuesta del humano.

4. **Reflexión.** Añade un cuarto nodo `criticar` que reciba el informe y lo evalúe, y una arista que vuelva a `redactar` si la crítica es negativa. Limita las vueltas a 2 con un contador. ¿Mejora el informe? ¿A qué precio en llamadas?

5. **Discusión para clase.** Con plan-and-execute, ¿quién es responsable si el plan está mal: el modelo o tú? ¿Cambia eso si el plan lo aprueba un humano?

05 · Ampliaciones (material para casa)

Este módulo no entra en las tres horas del taller. Son las cuatro piezas que pedirás **el lunes**, cuando intentes usar esto en algo real:

1. **Persistencia**: que la memoria sobreviva al reinicio.
2. **Streaming**: responder mientras el modelo escribe.
3. **Supervisión**: varios especialistas coordinados (el rol, a escala).
4. **Middleware**: resumir el historial, reintentos, filtros.

Al final tienes el reto integrador y la rúbrica de evaluación.

5.1 Persistencia: memoria en disco

`InMemorySaver` es estupendo para clase y **pésimo** para producción: al reiniciar el proceso, adiós conversaciones. Se cambia por un `checkpointer` que escribe en SQLite. Ni una línea más de código: es el mismo grafo.

Lo interesante didácticamente es **ver el fichero**. Así se entiende que «tener memoria» significa, literalmente, tener filas en una base de datos.

```python
import os
import sqlite3
from langgraph.graph import StateGraph, START, END, MessagesState
from langgraph.checkpoint.sqlite import SqliteSaver
from langchain_core.messages import SystemMessage
import comun

modelo = comun.llm()
RUTA = "memoria_seminario.db"

if os.path.exists(RUTA):
    os.remove(RUTA)                      # empezamos de cero para ver el efecto

conexion = sqlite3.connect(RUTA, check_same_thread=False)
checkpointer = SqliteSaver(conexion)

def nodo(estado: MessagesState):
    mensajes = [SystemMessage(content="Eres un chef conciso.")] + estado["messages"]
    return {"messages": [modelo.invoke(mensajes)]}

c = StateGraph(MessagesState)
c.add_node("agente", nodo)
c.add_edge(START, "agente")
c.add_edge("agente", END)

grafo_disco = c.compile(checkpointer=checkpointer)   # <-- unica diferencia
cfg = {"configurable": {"thread_id": "persistente-1"}}

grafo_disco.invoke({"messages": [{"role": "user", "content": "Me llamo Luis y me gusta el arroz."}]}, config=cfg)
respuesta = grafo_disco.invoke({"messages": [{"role": "user", "content": "¿Cómo me llamo?"}]}, config=cfg)
print("Respuesta:", respuesta["messages"][-1].content[:200])

print()
print("Y aquí está la memoria, en disco:")
tablas = [f[0] for f in conexion.execute("SELECT name FROM sqlite_master WHERE type='table'")]
print("  tablas          :", tablas)
filas = conexion.execute("SELECT count(*) FROM checkpoints").fetchone()[0]
print("  filas guardadas :", filas)
print("  tamaño fichero  :", os.path.getsize(RUTA), "bytes")
```

Ese fichero es la conversación. Si cierras el cuaderno, reinicias el kernel y vuelves a abrir la misma base de datos con el **mismo `thread_id`**, el hilo continúa donde estaba.

Aviso de concurrencia: SQLite va bien en clase y para un usuario. Con varios usuarios simultáneos necesitarás PostgreSQL (`langgraph-checkpoint-postgres`). Y recuerda que el `Store` de largo plazo es **otro** almacén: se persiste por separado (`langgraph.store.postgres.PostgresStore`), y es un fallo clásico persistir uno y olvidar el otro.

5.2 Streaming: no hacer esperar al usuario

Hasta ahora hemos usado `invoke`, que devuelve la respuesta completa. En una interfaz real quieres ver las palabras aparecer. `stream_events` da proyecciones tipadas: mensajes, valores, subgrafos e interrupciones.

```
peticion = {"messages": [{"role": "user", "content": "Explica en tres frases qué es un agente."}]}

flujo = grafo_disco.stream_events(peticion, config={"configurable": {"thread_id": "stream-1"}}, version="v3")

print("Llegando en directo:")
for mensaje in flujo.messages:
    for trozo in mensaje.text:
        print(trozo, end="", flush=True)
print()
```

Verás un `LangChainBetaWarning`: la API de eventos v3 todavía está marcada como experimental. Es la recomendada en la documentación, pero conviene saberlo antes de construir producción encima. La alternativa estable es `stream(..., stream_mode=[...], version="v2")`.

Consecuencia didáctica: si usas streaming, tu agente tarda lo mismo pero *parece* mucho más rápido. Es la mejora de experiencia de usuario más barata que existe.

5.3 Supervisión: el rol, a escala

Ya sabemos dar un rol a un agente. Un sistema multiagente es la misma idea repetida: varios especialistas y **alguien que decide a quién preguntar**. Ese alguien es el supervisor, y su decisión también es salida estructurada.

```
from typing import Annotated, Literal
from typing_extensions import TypedDict
import operator
from pydantic import BaseModel, Field
from langchain_core.messages import SystemMessage
from langgraph.graph import StateGraph, START, END

ESPECIALISTAS = {
    "chef": "Eres un chef mediterráneo. Propones platos sabrosos y rápidos.",
    "nutricionista": "Eres un nutricionista deportivo. Das cifras de calorías y macros.",
    "comprador": "Eres un experto en precios de supermercado. Hablas de costes y ahorro.",
}

class Decision(BaseModel):
    especialista: Literal["chef", "nutricionista", "comprador"] = Field(
        description="El especialista más adecuado para responder"
    )
    motivo: str = Field(description="Por qué ese y no otro")

class EstadoEquipo(TypedDict):
    peticion: str
    especialista: str
    motivo: str
    respuesta: str

enrutador = modelo.with_structured_output(Decision, method="function_calling")

```

```python
def nodo_supervisor(estado: EstadoEquipo):
    decision = enrutador.invoke(
        "Elige al especialista más adecuado para esta petición: " + estado["peticion"]
    )
    print(f"SUPERVISOR -> {decision.especialista}  ({decision.motivo})")
    return {"especialista": decision.especialista, "motivo": decision.motivo}

def nodo_especialista(estado: EstadoEquipo):
    sistema = SystemMessage(content=ESPECIALISTAS[estado["especialista"]])
    respuesta = modelo.invoke([sistema, ("human", estado["peticion"])])
    return {"respuesta": respuesta.content}

g = StateGraph(EstadoEquipo)
g.add_node("supervisor", nodo_supervisor)
g.add_node("especialista", nodo_especialista)
g.add_edge(START, "supervisor")
g.add_edge("supervisor", "especialista")
g.add_edge("especialista", END)
equipo = g.compile()

comun.ver_grafo(equipo, "Grafo 5: supervisor + especialista")
```

```python
for pregunta in [
    "¿Cuántas calorías tiene el pollo con arroz?",
    "¿Qué me cuesta la cena para cuatro?",
    "Quiero un plato resultón y rápido con lo que tengo.",
]:
    salida = equipo.invoke({"peticion": pregunta})
    print("PREGUNTA:", pregunta)
    print("RESPUESTA:", salida["respuesta"][:220].replace("\n", " "))
    print("-" * 70)
```

Este equipo es de un solo turno: el supervisor elige y el especialista responde. Convertirlo en un equipo que **itera** es un ejercicio de diez minutos con lo que ya sabes:

1. añade `"FIN"` a los valores posibles de `especialista`,
2. apunta las respuestas en una lista del estado (`Annotated[list, operator.add]`),
3. cambia la arista `especialista -> END` por `especialista -> supervisor`,
4. añade una arista condicional que vaya a `END` cuando el supervisor diga `"FIN"`,
5. pon un `recursion_limit` bajo (por ejemplo 12) para que no se desboque.

Ese es el patrón «supervisor» completo, y lo has construido tú.

5.4 Middleware: lo que no hemos escondido

LangChain 1.x trae *middleware* para los problemas transversales de cualquier agente. Los tres que necesitarás primero:

Middleware	Problema que resuelve
`SummarizationMiddleware`	el historial crece sin límite y se come el contexto
`HumanInTheLoopMiddleware`	aprobar herramientas peligrosas antes de ejecutarlas
`PIIMiddleware`	no dejar que datos personales acaben en el modelo

```python
from langchain.agents.middleware import SummarizationMiddleware

agente = create_agent(
    modelo, tools=HERRAMIENTAS,
    middleware=[SummarizationMiddleware(model=modelo, max_tokens_before_summary=4000)],
)
```

Los hemos dejado fuera a propósito: **primero la mecánica, después el atajo**. Si hubiéramos empezado por aquí, no sabrías qué se está resumiendo ni por qué.

Reto integrador (para casa o para evaluar)

Construye un **asistente de cocina con memoria y supervisión** que cumpla:

1. **Rol diferenciado.** Al menos dos roles con objetivos distintos y verificables.
2. **Herramientas.** Al menos tres, una de ellas con un modo de fallo controlado (`handle_tool_errors`) y un docstring que explique bien cuándo usarla.
3. **Memoria.** Corto plazo con `checkpointer` y largo plazo con `Store`, de modo que recuerde una alergia entre conversaciones distintas.
4. **Planificación.** Elige ReAct o plan-and-execute y **justifica** la elección por escrito, con el número de llamadas al modelo que hace cada uno en tu caso.
5. **Freno humano.** Un `interrupt()` antes de una acción con consecuencias.
6. **Verificación.** Al menos una `assert` que compruebe que el resultado cumple el contrato (por ejemplo, que la receta no lleva el ingrediente prohibido).

Rúbrica

Criterio	Insuficiente	Bien	Excelente
Rol	un prompt vago	rol con objetivo claro	rol con restricción verificada en código
Herramientas	devuelven datos fijos	docstrings claros y errores controlados	una herramienta consulta algo real
Memoria	solo `checkpointer`	corto y largo plazo funcionando	*namespaces* por usuario
Planificación	ReAct sin justificar	estrategia elegida y medida	compara ambas con datos
Freno humano	ausente	`interrupt` que funciona	con `response_schema` y aprobación del plan
Trazabilidad	solo la respuesta final	inspecciona `messages`	guarda trazas y las comenta

La pregunta que importa

Cuando entregues el reto, responde en cinco líneas:

> **¿En qué punto exacto de tu grafo tomaría una decisión equivocada el agente, y cómo te enterarías?**

Si sabes responder eso, has entendido el seminario. Si no, has aprendido a invocar una API.

Anexos

A. Evidencia de ejecución real

Las trazas siguientes están copiadas literalmente de las libretas una vez ejecutadas: no son ejemplos inventados. Es la prueba de que el taller funciona de principio a fin sobre Ollama.

A.1 Auditoría de modelos: retirados, fuera de cuota y utilizables

Salida del resumen de la libreta 00. Un catálogo de modelos caduca: de 24 modelos, solo 8 respondían el día de la verificación.

```
Modelos que RESPONDEN              : 8
Modelos RETIRADOS por el proveedor : 2   ['qwen3.5:cloud', 'qwen3.5:397b-cloud']
Modelos fuera del plan gratuito    : 11

Tres finales distintos, tres causas distintas:
  retirado  -> el proveedor lo quitó: hay que cambiar de modelo, no pagar
  sin cuota -> existe y funciona, pero no entra en tu plan: es dinero
  sano      -> adelante
```

A.2 Tool calling confirmado sobre el modelo gratuito

La prueba crítica del taller. Sin esto, ningún agente funciona.

```
======================================================================
PRUEBA CRÍTICA: ¿sabe este modelo usar herramientas?
======================================================================
tool_calls solicitados: [{'name': 'buscar_precio', 'args': {'producto': 'pollo'}, 'id': 'call_7fyw7rfc', 'type':
    'tool_call'}]

Tool calling confirmado. Todo listo para empezar.
```

A.3 La traza del bucle ReAct: el modelo pide, el grafo ejecuta

Obsérvese el `[01] AIMessage -> TOOL_CALLS`: el modelo no contesta, solicita dos herramientas en paralelo. Los `ToolMessage` llevan los datos reales calculados por Python.

```
PREGUNTA: ¿Cuánto cuesta el pollo? ¿Y cuánto son 100 EUR en USD?

[00] HumanMessage                ¿Cuánto cuesta el pollo? ¿Y cuánto son 100 EUR en USD?
[01] AIMessage  ->  TOOL_CALLS: buscar_precio({'producto': 'pollo'}), convertir_moneda({'cantidad': 100, 'destino
    ': 'USD', 'origen': 'EUR'})
[02] ToolMessage                 5.0
[03] ToolMessage                 110.0
[04] AIMessage                   El pollo cuesta 5.0 EUR y 100 EUR equivalen a 110.0 USD.

RESPUESTA FINAL:
El pollo cuesta 5.0 EUR y 100 EUR equivalen a 110.0 USD.
```

A.4 Memoria de largo plazo cruzando conversaciones distintas

Se guarda una alergia en el hilo 1 y se pregunta en el hilo 2, que es una conversación nueva. El `checkpointer` no puede ayudar: lo recuerda porque lo lee del `Store` mediante una herramienta.

```
HILO 1: Anotado. No incluiré frutos secos en ninguna de mis propuestas.
Almacén tras el hilo 1: {'alergia': 'frutos secos'}

HILO 2: Tienes alergia a los frutos secos.
```

```
5
6  Te propongo una cena ligera: **Salmón al horno con espárragos trigueros y patatas baby**. Simple, saludable y
       seguro para ti.
7
8  [00] HumanMessage                ¿Qué alergias tengo? Propón una cena.
9  [01] AIMessage  ->  TOOL_CALLS: leer_preferencias({})
10 [02] ToolMessage                 {'alergia': 'frutos secos'}
11 [03] AIMessage                   Tienes alergia a los frutos secos.  Te propongo una cena ligera: **Salmón al
       horno con espárragos trigueros y patatas baby**. Simple, saludable y seguro para ti.
```

A.5 Planificación explícita: el plan se lee antes de ejecutarse

Plan-and-execute en acción. La diferencia con ReAct no está en el modelo, sino en que el plan vive en el estado y es inspeccionable.

```
1  PLAN PROPUESTO:
2  1. Buscar el precio actual por kilogramo de pollo, arroz y espinacas en USD.
3  2. Calcular el costo total de 3 kg de pollo y la diferencia de precio entre el arroz y las espinacas.
4
5  --> ejecutando paso 1: Buscar el precio actual por kilogramo de pollo, arroz y espinacas en USD.
6  --> ejecutando paso 2: Calcular el costo total de 3 kg de pollo y la diferencia de precio entre el arroz y las
       espinacas.
7
8  ======================================================================
9  TRAZA DE RESULTADOS
10 ======================================================================
11 - Paso 1 - Buscar el precio actual por kilogramo de pollo, arroz y espinacas en USD. Los precios actuales por
       kilogramo son: - Pollo: 5.5 USD - Arroz: 2.2 USD - Espinacas: 3.3 USD
12
13 - Paso 2 - Calcular el costo total de 3 kg de pollo y la diferencia de precio entre el arroz y las espinacas. El
       costo total de 3 kg de pollo es de 15.0 EUR y la diferencia de precio entre el arroz y las espinacas es de
       1.
14
15 ======================================================================
16 INFORME FINAL
17 ======================================================================
18 Aquí tienes el informe redactado de manera clara y profesional:
19
20 ***
21
22 **INFORME DE COSTOS DE INSUMOS**
23
24 **1. Precios Unitarios (por kg)**
25 Se han registrado los siguientes precios actuales en USD:
26 *   **Pollo:** 5.5 USD
27 *   **Espinacas:** 3.3 USD
28 *   **Arroz:** 2.2 USD
29
30 **2. Cálculos Específicos**
31 *   **Costo de Pollo:** El valor total por 3 kg de pollo ascien
```

B. Verificación del entorno

Antes de empezar el taller se ejecuta `verificar_entorno.py`, que comprueba una por una todas las capacidades necesarias. Esta es su salida real:

```
================================================================================
VERIFICACIÓN DEL ENTORNO · Seminario de agentes con LangGraph
================================================================================

  comprobando: Daemon de Ollama accesible ...
  comprobando: Versiones de la pila ...
  comprobando: Modelo elegido y respondiendo ...
  comprobando: Tool calling (CRÍTICO) ...
  comprobando: Salida estructurada ...
  comprobando: Memoria de corto plazo ...
  comprobando: Memoria de largo plazo (Store) ...
  comprobando: Interrupción humana ...
  comprobando: Persistencia SQLite ...

================================================================================
RESULTADO
================================================================================
  [OK   ] Daemon de Ollama accesible       24 modelos en http://localhost:11434
  [OK   ] Versiones de la pila             langgraph 1.2.12 · langchain 1.4.3
  [OK   ] Modelo elegido y respondiendo    gemma4:31b-cloud (nube)
  [OK   ] Tool calling (CRÍTICO)           tool_calls: sonda
  [OK   ] Salida estructurada              Receta: Pollo al Limón y Ajo Express
  [OK   ] Memoria de corto plazo           4 mensajes en el checkpoint
  [OK   ] Memoria de largo plazo (Store)   escritura y lectura por namespace
  [OK   ] Interrupción humana              pausa y reanudación con Command(resume=...)
  [OK   ] Persistencia SQLite              langgraph-checkpoint-sqlite disponible

  Todo correcto: el seminario puede empezar.
```

C. Referencia rápida de la API

Las rutas siguientes están verificadas contra los paquetes publicados (`langgraph` 1.2.12, `langchain` 1.4.3, `langchain-core` 1.6.6). Es la API que usan las seis libretas del taller.

Para qué	Importación y firma
Grafo y estado	`from langgraph.graph import StateGraph, START, END, MessagesState`
Agente prebuilt	`from langchain.agents import create_agent`
Herramientas	`from langchain.tools import tool, ToolRuntime`
Nodo de herramientas	`from langgraph.prebuilt import ToolNode, tools_condition`
Memoria de conversación	`from langgraph.checkpoint.memory import InMemorySaver`
Memoria de largo plazo	`from langgraph.store.memory import InMemoryStore`
Pausa humana	`from langgraph.types import interrupt, Command`
Persistencia en disco	`from langgraph.checkpoint.sqlite import SqliteSaver`

El grafo mínimo. Un nodo recibe el estado y devuelve *solo* la clave que modifica; LangGraph la fusiona con el estado anterior.

```
constructor = StateGraph(MessagesState)
constructor.add_node("agente", nodo)
constructor.add_edge(START, "agente")
constructor.add_edge("agente", END)
grafo = constructor.compile(checkpointer=InMemorySaver())
```

El bucle ReAct. La arista condicional es el bucle, y hay que mapear la salida de `tools_condition`, que devuelve los literales `"tools"` y `"__end__"`:

```
constructor.add_conditional_edges(
    "agente", tools_condition, {"tools": "herramientas", "__end__": END}
)
constructor.add_edge("herramientas", "agente")
```

El agente completo en una línea, con memoria y almacén:

```
agente = create_agent(
    modelo,
    tools=[buscar_precio, convertir_moneda],
    system_prompt="Eres un chef conciso.",
    response_format=Receta,          # salida estructurada garantizada
    checkpointer=InMemorySaver(),
    store=InMemoryStore(),
)
resultado = agente.invoke(
    {"messages": [{"role": "user", "content": "¿Cuánto cuesta el pollo?"}]},
    config={"configurable": {"thread_id": "hilo-1"}},
)
```

Intervención humana. Requiere `checkpointer`; el nodo se reejecuta entero al reanudar, así que los efectos secundarios deben ser idempotentes.

```python
def nodo(estado):
    decision = interrupt({"pregunta": "¿Aprobamos el gasto?"})
    return {"messages": [AIMessage(content=f"Decisión: {decision}")]}

parada = grafo.invoke(entrada, config=cfg)      # se detiene aquí
final  = grafo.invoke(Command(resume="sí"), config=cfg)   # mismo thread_id
```

Advertencias que ahorran tiempo

Síntoma	Causa real
`found unknown target 'tools'`	`tools_condition` devuelve `"tools"`; falta el mapa
La memoria «no funciona» sin error	Falta `checkpointer=` o cambia el `thread_id`
`Missing required config key`	Un `ToolNode` invocado fuera de un grafo
El agente se inventa los datos	No hubo `tool_calls` en la traza
`with_structured_output` devuelve prosa	Usa `method="function_calling"`
`tool_choice` se ignora	Ollama no lo soporta
`HTTP 402` / `HTTP 410`	Modelo fuera del plan gratuito / retirado

No se enseña (y por qué): `initialize_agent` y `AgentType` (obsoletos desde LangChain 1.0), `create_react_agent` (emite `LangGraphDeprecationWarning` y remite a `langchain.agents.create_agent`), `MemorySaver` (alias de `InMemorySaver`), ni las rutas antiguas `langchain.prompts` / `langchain.output_parsers`.

D. Estructura del material

```
seminario/
.env.example
00_preflight.ipynb
01_rol.ipynb
02_herramientas.ipynb
03_memoria.ipynb
04_planificacion.ipynb
05_ampliaciones.ipynb
README.md
_build/
    |-- build.py
    |-- nb_00.py
    |-- nb_01.py
    |-- nb_02.py
    |-- nb_03.py
    |-- nb_04.py
    |-- nb_05.py
    |-- pdf.py
comun.py
pdf/
    |-- a-52.png
    |-- c-19.png
    |-- logo_esi.png
    |-- logo_uclm.png
    |-- pag-1.png
    |-- plantilla.tex
    |-- prueba.tex
    |-- v-01.png
    |-- v-02.png
    |-- v-03.png
    |-- v-14.png
    |-- vista.png
    |-- vista2.png
referencia/
    |-- api-2026.md
    |-- ollama-cloud-2026.md
requirements.txt
verificar_entorno.py
```